

AURORA-01 MISSION CONTROL SIMULATOR

A Python-Based Aerospace Mission Control Simulation

Project Report

Submitted as part of
VITyarthi – Build Your Own Project

Subject: Python Essentials / Programming

Project Type: Command-Line Application

Submitted By:

Name: Srushti Mahesh Joshi

Roll No.: 26BAS10062

Slot: C11+C12+C13+F11+F12+F14

Class: First Year Engineering

Academic Year: 2026–27

Institution: VIT BHOPAL

Department: Aerospace engineering

ABSTRACT

AURORA-01 Mission Control Simulator is a Python-based command-line simulation designed to demonstrate the basic operations of an aerospace mission control system. The project allows users to monitor mission telemetry such as fuel level, oxygen level, temperature, battery level, and altitude.

The simulator provides system diagnostics, emergency situation simulation, and automated mission decisions based on telemetry conditions. The possible mission decisions are **GO**, **CAUTION**, and **ABORT**. An event logging system records important mission activities, and mission data can be saved for later reference.

The project demonstrates the practical application of Python programming concepts including variables, data types, dictionaries, conditional statements, functions, modules, exception handling, file handling, and the use of the datetime module. The system is developed as an educational simulation and is not intended for real spacecraft control.

1. INTRODUCTION

Aerospace missions require continuous monitoring of important spacecraft parameters and appropriate decisions when abnormal conditions occur. A mission control system helps monitor these parameters and supports the process of identifying possible problems.

AURORA-01 Mission Control Simulator is a Python-based command-line application developed as an educational simulation of such a system. It allows the user to start a mission, enter and monitor spacecraft telemetry, perform system diagnostics, simulate emergency conditions, and generate a mission decision.

The project uses Python programming concepts such as variables, dictionaries, conditional statements, functions, modules, loops, exception handling, file handling, and event logging. The modular structure separates different responsibilities into individual Python files, making the project easier to understand and maintain.

The simulator is designed for learning and demonstration purposes. It does not represent or control a real spacecraft system.

2. PROBLEM STATEMENT

Aerospace missions require continuous monitoring of critical parameters such as fuel, oxygen, temperature, battery level, and altitude. Abnormal changes in these parameters may require immediate attention and appropriate decisions.

For students learning Python programming, understanding how such monitoring and decision-making systems work can be difficult using only theoretical examples.

Therefore, the **AURORA-01 Mission Control Simulator** is developed as an educational Python application that simulates basic mission-control operations. The system accepts telemetry data, performs system diagnostics, simulates emergency conditions, generates mission decisions such as **GO, CAUTION, and ABORT**, and maintains an event log.

The project provides a practical application of Python programming concepts while demonstrating how a modular software system can be designed for a specific problem.

3. OBJECTIVES

The main objectives of the AURORA-01 Mission Control Simulator are:

1. To develop a Python-based simulation of a basic aerospace mission control system.
2. To monitor important mission telemetry such as fuel, oxygen, temperature, battery level, and altitude.
3. To perform system diagnostics and identify warning or critical conditions.
4. To simulate common emergency situations and observe their effect on mission parameters.
5. To generate automated mission decisions such as **GO, CAUTION, and ABORT** based on telemetry conditions.
6. To maintain an event log of important mission activities.
7. To demonstrate practical implementation of Python concepts using a modular program structure.
8. To provide an educational simulation for students learning Python and basic aerospace-system concepts.

4. FUNCTIONAL REQUIREMENTS

The AURORA-01 Mission Control Simulator shall provide the following functional requirements:

1. **Mission Initialization**

The system shall allow the user to start the AURORA-01 mission.

2. **Telemetry Monitoring**

The system shall accept and display mission telemetry including fuel, oxygen, temperature, battery level, and altitude.

3. **System Diagnostics**

The system shall analyze telemetry values and identify normal, warning, and critical conditions.

4. **Emergency Simulation**

The system shall allow the user to simulate emergency conditions such as fuel leak, oxygen loss, overheating, and battery failure.

5. **Mission Decision**

The system shall generate a mission decision based on the current telemetry conditions. The possible decisions are **GO, CAUTION, and ABORT**.

6. **Event Logging**

The system shall record important mission activities with timestamps.

7. **Mission Data Saving**

The system shall allow mission information and event data to be saved to a file.

8. **Mission Summary**

The system shall display a summary of the mission status and important mission information

5. NON-FUNCTIONAL REQUIREMENTS

The AURORA-01 Mission Control Simulator shall satisfy the following non-functional requirements:

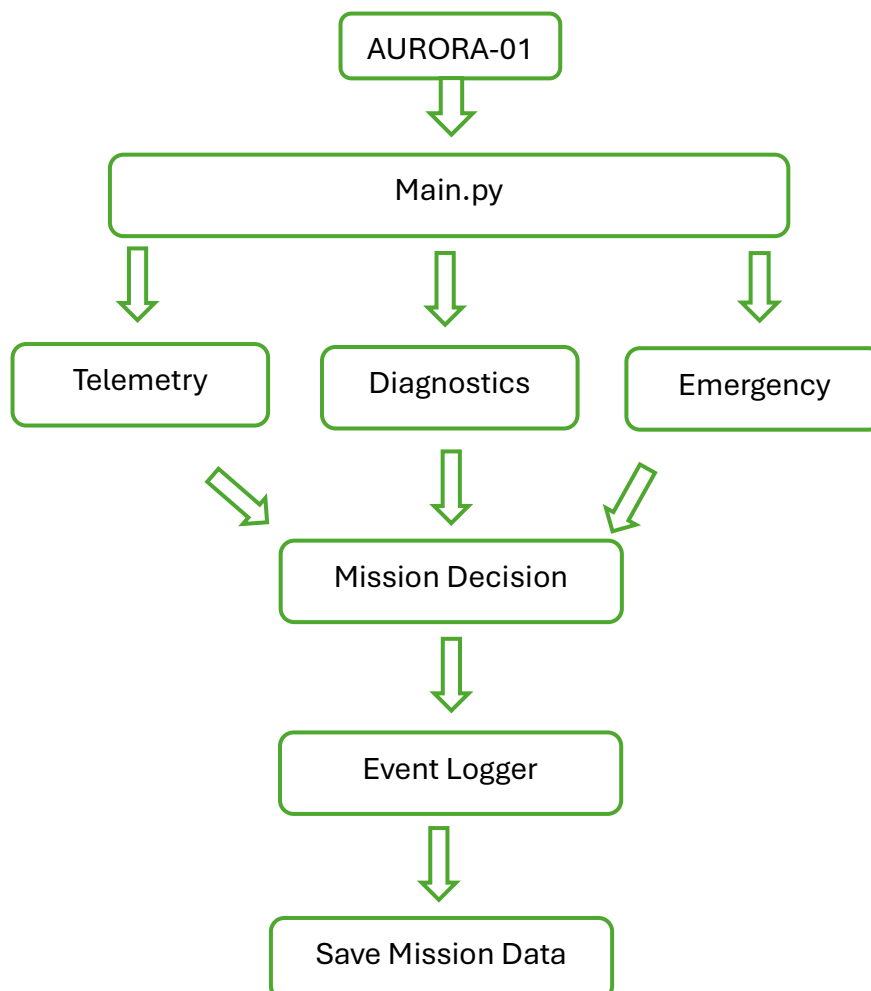
1. **Usability** – The system should provide a simple and clear command-line interface so that beginners can easily operate it.
2. **Reliability** – The system should handle invalid user inputs without unexpectedly terminating the program.
3. **Maintainability** – The program should use separate modules for different functions, making the code easier to understand, modify, and maintain.
4. **Error Handling** – The system should validate user inputs and handle common input errors using exception handling.
5. **Performance** – The system should process telemetry, diagnostics, and mission decisions quickly without unnecessary resource usage.
6. **Scalability** – The modular structure should allow additional telemetry parameters, emergency scenarios, and mission features to be added in the future.
7. **Logging** – Important mission activities should be recorded with timestamps for tracking and review.
8. **Portability** – The application should run using standard Python without requiring external libraries.

6. SYSTEM ARCHITECTURE

The AURORA-01 Mission Control Simulator follows a modular architecture in which different mission-control responsibilities are separated into individual Python modules.

The **main.py** module acts as the central controller and connects the different modules. The **telemetry.py** module handles telemetry input and display, while **diagnostics.py** performs system diagnostics. The **emergency.py** module handles emergency simulations, and **mission_decision.py** determines the mission status. The **logger.py** module records important mission events with timestamps.

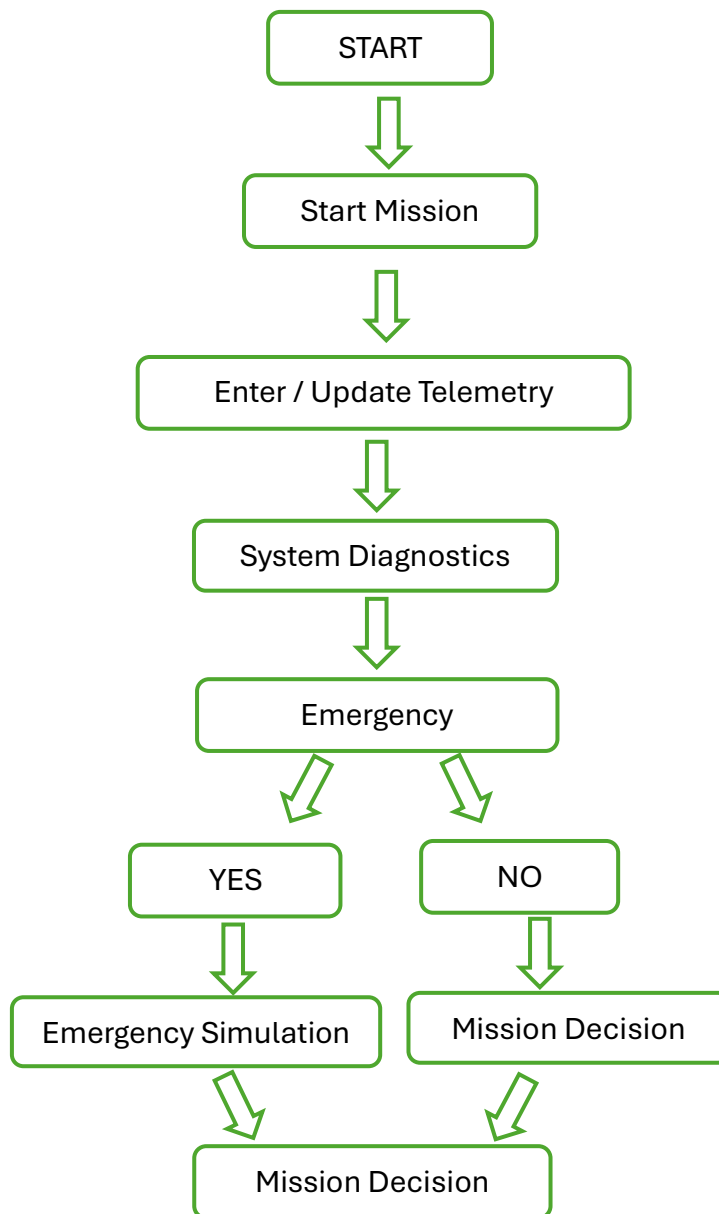
This modular architecture improves code organization, readability, maintainability, and future extensibility.

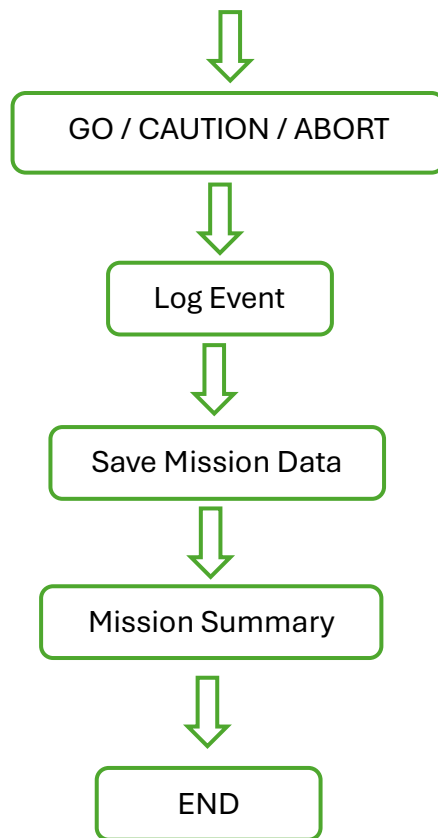


7. DESIGN DIAGRAMS

7.1 Workflow Diagram

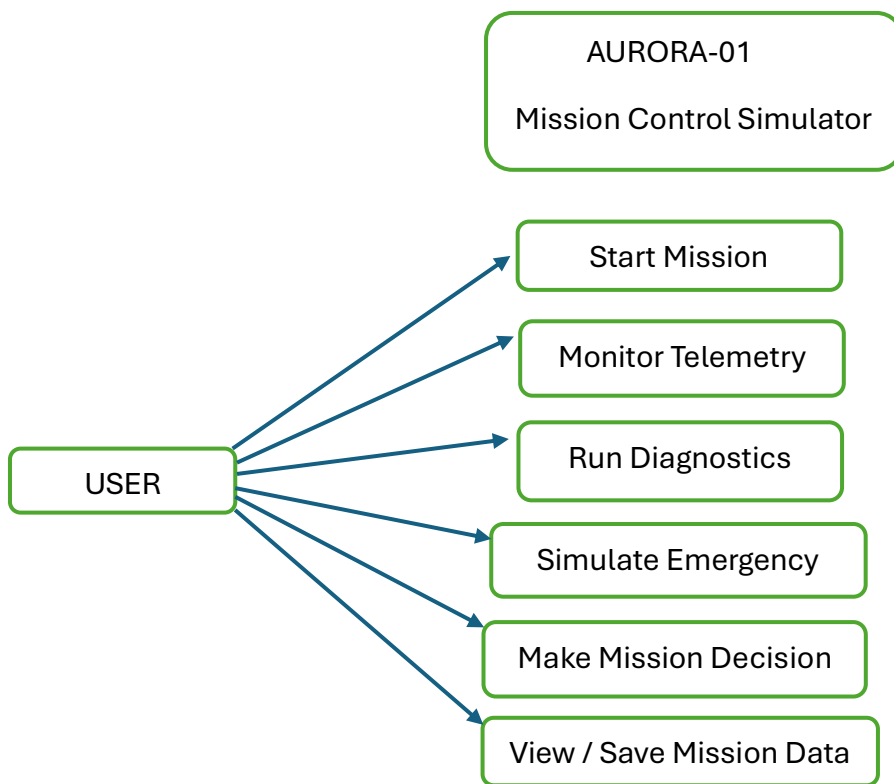
The workflow diagram represents the sequence of operations performed by the AURORA-01 Mission Control Simulator, starting from mission initialization and continuing through telemetry monitoring, diagnostics, emergency simulation, mission decision-making, event logging, and mission summary.





7.2 USE CASE DIAGRAM

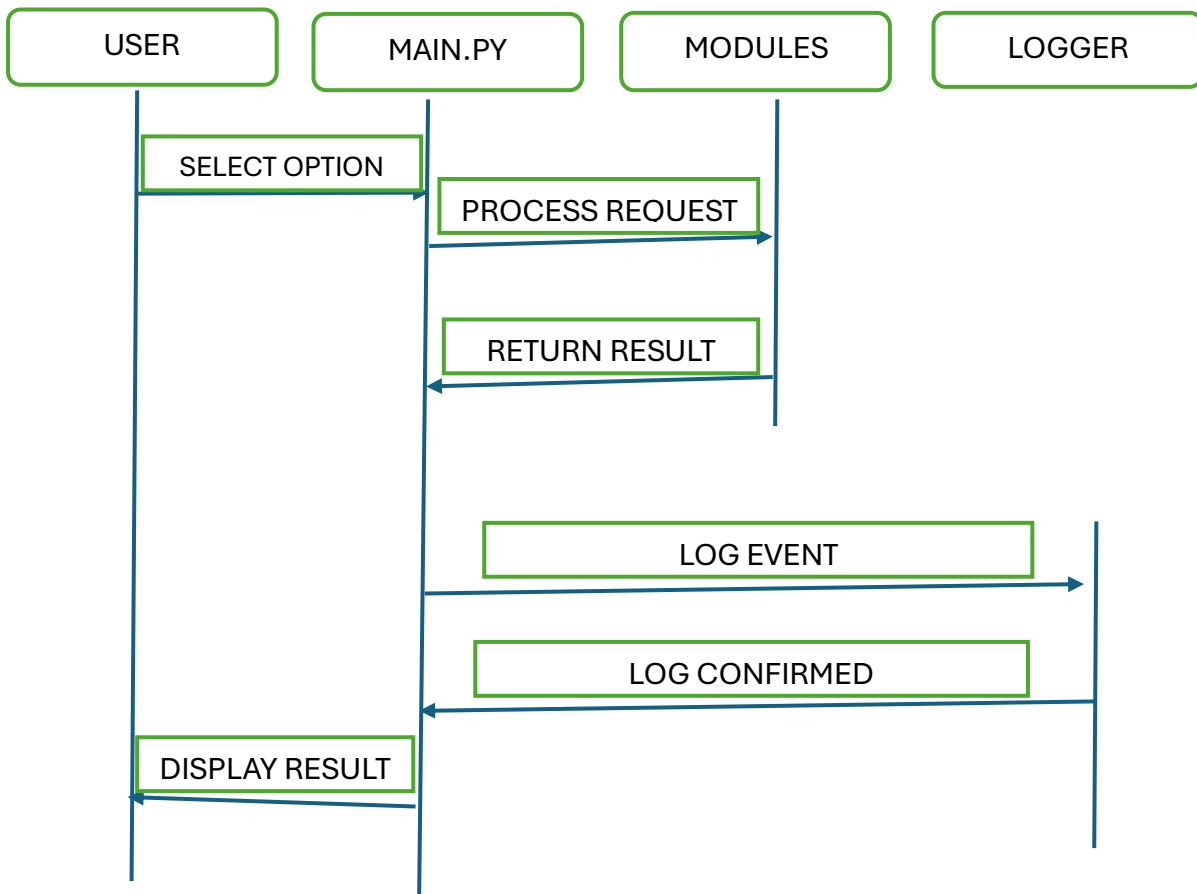
The Use Case Diagram shows the interaction between the user and the major functions of the AURORA-01 Mission Control Simulator. The user can start the mission, monitor telemetry, perform system diagnostics, simulate emergencies, generate mission decisions, view the event log, save mission data, and view the mission summary.



7.3 SEQUENCE DIAGRAM

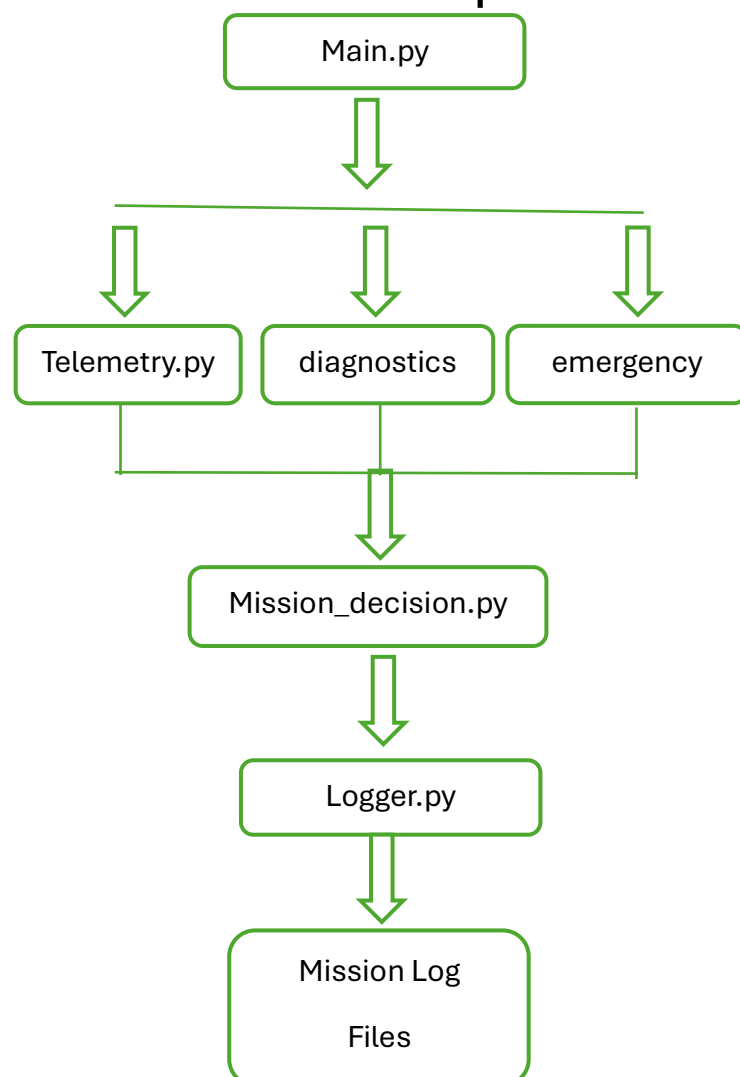
The Sequence Diagram represents the interaction between the user, the main program, the functional modules, and the event logger during the execution of the simulator. The user provides commands and telemetry data through the main program. The main program communicates with the appropriate modules for telemetry processing, diagnostics, emergency simulation, and mission decision-making. Important activities are recorded by the logger.

SEQUENCE DIAGRAM



7.4 COMPONENT DIAGRAM

The Component Diagram represents the modular structure of the AURORA-01 Mission Control Simulator. Each Python module performs a specific responsibility. The main program coordinates the modules, while the telemetry, diagnostics, emergency, mission decision, and logger modules work together to perform the required mission-control operations.



8. DESIGN DECISIONS & RATIONALE

The following design decisions were made while developing the AURORA-01 Mission Control Simulator:

1. Modular Program Structure

The project is divided into multiple Python modules, with each module responsible for a specific task. This improves readability, maintainability, and organization.

2. Command-Line Interface

A command-line interface was selected because it keeps the project simple and focuses on demonstrating Python programming concepts and logical decision-making.

3. Dictionary for Telemetry Data

A Python dictionary is used to store telemetry parameters such as fuel, oxygen, temperature, battery, and altitude. This provides a simple way to organize and access mission data.

4. Threshold-Based Decision Making

Predefined threshold values are used to classify telemetry conditions as normal, warning, or critical and generate the corresponding mission decision.

5. Event Logging

A separate logger module is used to record important mission events with timestamps. This makes mission activities easier to track and review.

6. Standard Python Libraries

The project uses Python's standard library, including the datetime module, so that no external package installation is required.

9. IMPLEMENTATION DETAILS

The AURORA-01 Mission Control Simulator is implemented using Python and follows a modular programming approach. The main program coordinates the different modules and provides a menu-driven interface for the user.

1. **main.py**

Acts as the central controller of the application. It displays the main menu and coordinates the different mission-control operations.

2. **telemetry.py**

Handles the input and display of mission telemetry parameters such as fuel, oxygen, temperature, battery, and altitude.

3. **diagnostics.py**

Checks telemetry values against predefined thresholds and identifies warning or critical conditions.

4. **emergency.py**

Provides options to simulate emergency situations such as fuel leak, oxygen loss, overheating, and battery failure.

5. **mission_decision.py**

Analyses the current telemetry conditions and generates one of three mission decisions: **GO, CAUTION, or ABORT.**

6. **logger.py**

Maintains an event log and records important mission activities along with timestamps.

7. **File Handling**

Mission information and event data can be saved to a text file for later reference.

8. **Exception Handling**

Input validation and exception handling are used to prevent invalid user input from unexpectedly terminating the application.

The implementation uses Python concepts including variables, dictionaries, conditional statements, loops, functions, modules, exception handling, file handling, and the datetime module.

10. SCREENSHOTS & RESULTS

The AURORA-01 Mission Control Simulator was executed successfully using Python IDLE. The following screenshots demonstrate the main interface, telemetry monitoring, emergency simulation, and mission decision functionality.

10.1 Main Menu



```
*IDLE Shell 3.9.7*
File Edit Shell Debug Options Window Help
Python 3.9.7 (tags/v3.9.7:1016ef3, Aug 30 2021, 20:19:38) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\HP\AppData\Local\Programs\Python\Python39\Aurora mission control.py =====
=====
AURORA-01 MISSION CONTROL
Monitor. Diagnose. Decide. Survive.
=====
Mission Status : STANDBY

-----
1. Start Mission
2. View Telemetry
3. Run Diagnostics
4. Simulate Emergency
5. Mission Decision
6. View Event Log
7. Save Mission Data
8. Mission Summary
9. Exit
-----
Enter command: |
```

10.2 Telemetry Monitoring

```
*IDLE Shell 3.9.7*
File Edit Shell Debug Options Window Help
RESTART: C:\Users\HP\AppData\Local\Programs\Python\Python39\Aurora mission control.py

=====
AURORA-01 MISSION CONTROL
Monitor. Diagnose. Decide. Survive.
=====

Mission Status : STANDBY

-----
1. Start Mission
2. View Telemetry
3. Run Diagnostics
4. Simulate Emergency
5. Mission Decision
6. View Event Log
7. Save Mission Data
8. Mission Summary
9. Exit
-----
Enter command: 1

=====
MISSION STARTED
=====
Mission ID : AURORA-01
Mission Status : ACTIVE
Crew Status : READY
Communication : ONLINE

AURORA-01 is now active.
Press ENTER to continue...

=====
AURORA-01 MISSION CONTROL
Monitor. Diagnose. Decide. Survive.
=====

Mission Status : ACTIVE

-----
1. Start Mission
2. View Telemetry
3. Run Diagnostics
4. Simulate Emergency
5. Mission Decision
6. View Event Log
7. Save Mission Data
8. Mission Summary
9. Exit
-----
Enter command:
```

10.3 Emergency Simulation and Mission Decision

```
*IDLE Shell 3.9.7*
File Edit Shell Debug Options Window Help

-----
1. Start Mission
2. View Telemetry
3. Run Diagnostics
4. Simulate Emergency
5. Mission Decision
6. View Event Log
7. Save Mission Data
8. Mission Summary
9. Exit
-----
Enter command: 3

[SYSTEM DIAGNOSTICS]
-----
Fuel System : OK
Oxygen System : OK
Temperature : OK
Battery System : OK
Communication : ONLINE

Diagnostics completed.
Press ENTER to continue...

=====
AURORA-01 MISSION CONTROL
Monitor. Diagnose. Decide. Survive.
=====

Mission Status : ACTIVE

-----
1. Start Mission
2. View Telemetry
3. Run Diagnostics
4. Simulate Emergency
5. Mission Decision
6. View Event Log
7. Save Mission Data
8. Mission Summary
9. Exit
-----
Enter command: 4

[EMERGENCY SIMULATION]
-----
1. Fuel Leak
2. Oxygen Loss
3. Overheating
4. Battery Failure
5. Cancel
Select emergency: 2
```

11. TESTING APPROACH

The AURORA-01 Mission Control Simulator was tested by executing different functional scenarios and checking whether the system produced the expected output.

Test Case	Input / Scenario	Expected Result
TC01	Start Mission	Mission starts successfully
TC02	Enter valid telemetry values	Telemetry is accepted and displayed
TC03	Enter invalid telemetry input	Error message is displayed and program continues
TC04	Normal telemetry conditions	Diagnostic status is normal
TC05	Low fuel / oxygen condition	Warning or critical condition is detected
TC06	Simulate oxygen loss	Oxygen level decreases and event is logged
TC07	Critical telemetry condition	Mission decision becomes ABORT
TC08	Normal telemetry conditions	Mission decision becomes GO
TC09	Warning telemetry conditions	Mission decision becomes CAUTION

Test Case	Input / Scenario	Expected Result
TC10	Save Mission Data	Mission information is saved to a text file

The testing confirmed that the major functional modules of the simulator operate according to the intended workflow.

12. CHALLENGES FACED

During the development of the AURORA-01 Mission Control Simulator, several challenges were encountered:

1. Understanding Modular Programming

Dividing the project into separate Python modules and managing communication between them required careful planning.

2. Telemetry Validation

Handling user-entered telemetry values and preventing invalid inputs required input validation and exception handling.

3. Mission Decision Logic

Defining appropriate threshold conditions for **GO**, **CAUTION**, and **ABORT** required careful implementation of conditional statements.

4. Emergency Simulation

Designing emergency scenarios so that they correctly affected the corresponding telemetry parameters required additional testing.

5. Event Logging

Maintaining a useful event history with timestamps required the implementation of a separate logging module.

6. Documentation and System Design

Creating architecture, workflow, UML diagrams, screenshots, and documentation required understanding the complete flow of the application.

13. LEARNINGS & KEY TAKEAWAYS

The development of the AURORA-01 Mission Control Simulator provided practical experience in applying Python programming concepts to a real-world-inspired problem.

The major learnings from the project include:

1. Understanding how to convert a problem statement into a functional software solution.
2. Applying Python variables, data types, dictionaries, operators, conditional statements, loops, and functions.
3. Understanding modular programming and dividing a program into separate functional modules.
4. Using input validation and exception handling to make the application more reliable.
5. Applying conditional logic for automated mission decision-making.
6. Implementing event logging using timestamps.
7. Using file handling to save mission information.
8. Understanding the importance of software architecture and design diagrams.

9. Learning how testing helps identify and verify different system behaviours.
10. Gaining experience in documenting and presenting a complete software project.

14. FUTURE ENHANCEMENTS

The AURORA-01 Mission Control Simulator can be further enhanced with additional features in future versions:

- 1. Graphical User Interface (GUI)**

Develop a graphical dashboard to display telemetry and mission status visually.

- 2. Real-Time Telemetry Simulation**

Add continuously changing telemetry values to simulate a more realistic mission environment.

- 3. Additional Emergency Scenarios**

Include more emergency conditions and their corresponding effects on mission parameters.

- 4. Telemetry Graphs and Charts**

Display changes in fuel, oxygen, temperature, battery, and altitude using graphs.

- 5. Database Integration**

Store mission data and event logs in a database instead of only using text files.

- 6. Automated Test Suite**

Add automated tests to verify individual modules and system behaviour.

7. Advanced Mission Simulation

Add additional mission phases, spacecraft parameters, and more detailed mission-control logic.

15. REFERENCES

1. VITyarthi, **Build Your Own Project – Project Guidelines**, provided course/project guidelines document.
2. Python Software Foundation, **Python Documentation**, Python 3.x documentation.
3. VITyarthi course material and learning resources related to Python programming.
4. Project source code and design documentation developed for **AURORA-01 Mission Control Simulator**.